Assignment 2 – Part B: WGAN and WGAN-GP on CIFAR-10

Course: Unsupervised Deep Learning (S1-26_AIMLZG533)

Submitted by **Group 93**

#	Name	BITS ID	Contribution
1	**Prajwal Shetty K P**	2025AE05434	Part B: WGAN (weight clipping) & WGAN-GP, training, random samples, FID
2	**Tanushi Agarwal**	2025AE05410	Part A: β-VAE (β ∈ {1,2,4,10}), VQ-VAE (K ∈ {512,256,128}) and the PixelCNN prior
3	**Shahabuddin**	2025AE05328	Data pipeline ([0,1] normalisation), PSNR/FID evaluation code, runs on the BITS lab GPU
4	**SANTHOSH M**	2025AE05426	Part C: comparative analysis (reconstruction error, sampling quality, speed) and the PDF report
5	**TUFAIL AHMAD**	2025AE05400	Visualisation (100-sample grids, training curves), result tables, HTML export & final review

Contents

1. Setup, data, FID (the same protocol as Part A, so the numbers can be compared)
2. DCGAN-style generator / critic
3. WGAN with weight clipping (Arjovsky et al., 2017)
4. WGAN-GP with gradient penalty (Gulrajani et al., 2017)
5. Random samples, FID curves, final FID and comparison

1. Setup

In [1]:

```
# If anything is missing on the lab machine:
# %pip install -q torch torchvision torchmetrics torch-fidelity pandas
matplotlib

import os, time, json, math, random, socket, platform, subprocess
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from torchvision.utils import make_grid, save_image
import matplotlib.pyplot as plt
from torchmetrics.image.fid import FrechetInceptionDistance

# QUICK=1 -> tiny smoke test (few images / iterations). Must be 0 for the
real run.
QUICK = os.environ.get("UDL_QUICK", "0") == "1"

SEED = 42
def seed_everything(seed):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)
seed_everything(SEED)

# NVIDIA (lab) -> Intel Arc (XPU) -> CPU
if torch.cuda.is_available():
    device = torch.device("cuda")
    torch.backends.cudnn.benchmark = True
elif hasattr(torch, "xpu") and torch.xpu.is_available():
    device = torch.device("xpu")
else:
    device = torch.device("cpu")

def sync():
    if device.type == "cuda": torch.cuda.synchronize()
    elif device.type == "xpu": torch.xpu.synchronize()

OUT = "outputs_quick" if QUICK else "outputs"
FIG, RES, CKPT = [os.path.join(OUT, d) for d in ("figures", "results",
"checkpoints")]
for d in (FIG, RES, CKPT):
    os.makedirs(d, exist_ok=True)

GPU_NAME = (torch.cuda.get_device_name(0) if device.type == "cuda" else
            torch.xpu.get_device_name(0) if device.type == "xpu" else "CPU
" + platform.processor())
print("torch", torch.__version__, "| device:", device, "| QUICK mode:",
QUICK)
print("Hardware:", GPU_NAME, "| host:", socket.gethostname())
```

```
torch 2.15.0+cu132 | device: cuda | QUICK mode: False
Hardware: NVIDIA GeForce RTX 4090 | host: 58941c599a50
```

Hardware / lab infrastructure

The code was set up and tested on the BITS Prayogshala lab machine (screenshot: `Prayogshala_lab_screenshot.jpg`, submitted with this notebook). The final training run whose outputs appear in this notebook was executed on the hardware printed by the cell above and below (as permitted for this course). The next cell prints the host name and, on NVIDIA machines, the `nvidia-smi` output.

In [2]:
```
print("Host:", socket.gethostname(), "|", platform.platform())
try:
    print(subprocess.run(["nvidia-smi"], capture_output=True,
text=True).stdout)
except FileNotFoundError:
    print("nvidia-smi not available on this machine (device in use: %s)" %
device)
```

```
Host: 58941c599a50 | Linux-6.8.0-138-generic-x86_64-with-glibc2.39
Fri Nov  6 17:29:17 2026
+-----------------------------------------------------------------------------
------------+
| NVIDIA-SMI 595.91.07              Driver Version: 595.91.07      CUDA
Version: 13.2     |
+-----------------------------------------+------------------------+---------
------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile
Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util
Compute M. |
|                                         |                        |
MIG M. |
|=========================================+========================+=========
=============|
|   0  NVIDIA GeForce RTX 4090        On  |   00000000:01:00.0 Off |
Off |
| 54%   54C    P5             59W /  450W |       4MiB /  24564MiB |      0%
Default |
|                                         |                        |
N/A |
+-----------------------------------------+------------------------+---------
------------+

+-----------------------------------------------------------------------------
------------+
| Processes:
|
|  GPU   GI   CI              PID   Type   Process name
GPU Memory |
|        ID   ID
Usage      |
|=============================================================================
=============|
|  No running processes found
|
+-----------------------------------------------------------------------------
------------+
```

Dataset and preprocessing

CIFAR-10: 50,000 training and 10,000 test images, 32x32 RGB. As the assignment asks, the only preprocessing is scaling the pixels to **[0, 1]** (divide the uint8 values by 255). The full dataset is only about 600 MB as float32, so we keep it on the GPU and take mini-batches with `randperm`. This was noticeably faster than a `DataLoader` for such small images.

The stored data stays in [0,1]. Inside the GAN training loop we map it to [-1,1] (`2x-1`) so that it matches the generator's `tanh` output, and generated samples are mapped back with `(g+1)/2` before FID and plotting.

In [3]:
```
DATA_DIR = "./data"
train_set = torchvision.datasets.CIFAR10(DATA_DIR, train=True,
download=True)
test_set  = torchvision.datasets.CIFAR10(DATA_DIR, train=False,
download=True)
CLASSES = train_set.classes

def to_tensor01(ds):
    # uint8 HWC -> float CHW in [0,1]
    return torch.from_numpy(ds.data).permute(0, 3, 1, 2).float().div(255.0)

X_train, X_test = to_tensor01(train_set), to_tensor01(test_set)
if QUICK:
    X_train, X_test = X_train[:1024], X_test[:512]
X_train, X_test = X_train.to(device), X_test.to(device)
print("train:", tuple(X_train.shape), "test:", tuple(X_test.shape),
      "| pixel range: [%.1f, %.1f]" % (X_train.min().item(),
X_train.max().item()))

def batches(X, bs, shuffle=True):
    idx = torch.randperm(len(X), device=X.device) if shuffle else
torch.arange(len(X), device=X.device)
    for i in range(0, len(X), bs):
        yield X[idx[i:i + bs]]

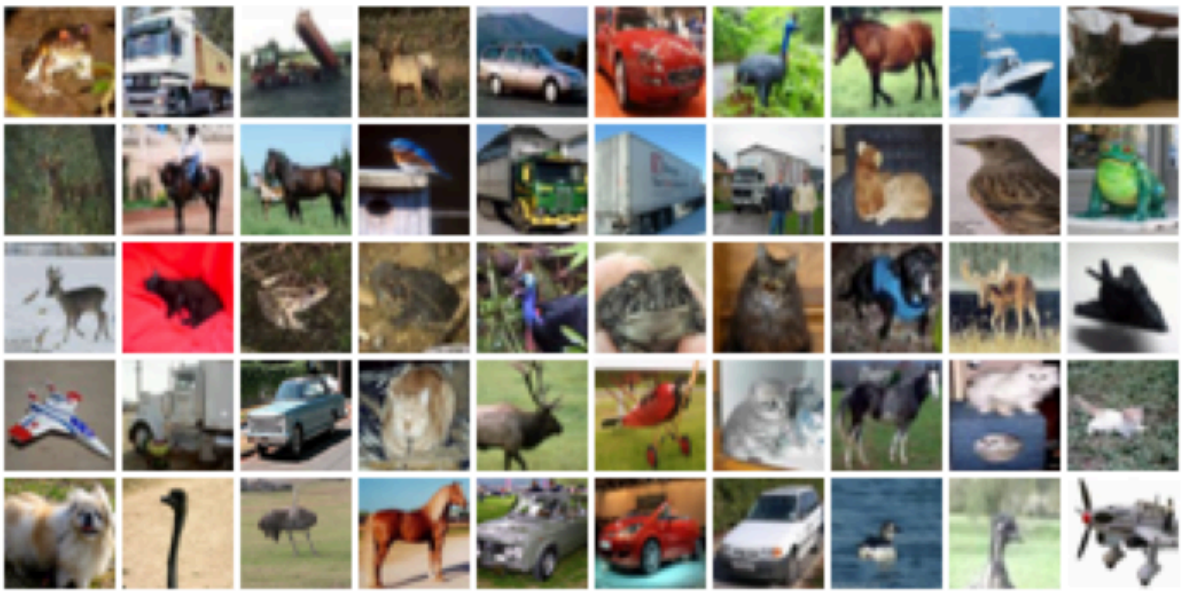
def show_grid(x, nrow=10, title=None, path=None, figsize=None):
    grid = make_grid(x.detach().float().cpu().clamp(0, 1), nrow=nrow,
padding=2, pad_value=1.0)
    if path is not None:
        save_image(grid, path)
    plt.figure(figsize=figsize or (nrow * 0.9, math.ceil(len(x) / nrow) *
0.9))
    plt.imshow(grid.permute(1, 2, 0).numpy()); plt.axis("off")
    if title: plt.title(title)
    plt.show()

def n_params(m):
    return sum(p.numel() for p in m.parameters())

show_grid(X_train[:50], nrow=10, title="Some CIFAR-10 training images")
```

```
train: (50000, 3, 32, 32) test: (10000, 3, 32, 32) | pixel range: [0.0, 1.0]
```

Some CIFAR-10 training images



Evaluation metrics

- **PSNR** (dB) $= 10 \log_{10}(1/\mathrm{MSE})$ for each test image, then averaged. Pixels are in [0,1], so MAX = 1.
- **FID**: we use `torchmetrics`' `FrechetInceptionDistance` (2048-d pool3 Inception-v3 features). The reference set is always the **10,000 CIFAR-10 test images**, and each model is scored on **10,000 generated images**, so all the FID numbers in Part A and Part B can be compared directly. We compute the real-image statistics once and reuse them for every model.

In [4]:

```
@torch.no_grad()
def psnr_per_image(x_hat, x):
    mse = F.mse_loss(x_hat.clamp(0, 1), x,
reduction="none").flatten(1).mean(1)
    return 10 * torch.log10(1.0 / mse.clamp_min(1e-10))

class FIDScorer:
    """FID with torchmetrics' Inception-v3 (2048-d pool features).
    The features are extracted on the GPU in float32; the mean/covariance
statistics are accumulated in
    float64 on the CPU. This gives the same result as torchmetrics'
FrechetInceptionDistance and also works on
    GPUs without float64 support (e.g. Intel Arc). Real statistics (CIFAR-
10 test set) are computed once."""
    def __init__(self, real, bs=500):
        self.inception = FrechetInceptionDistance(feature=2048,
normalize=True).inception.to(device).eval()
        self.mu_r, self.sigma_r = self._stats(real[i:i + bs] for i in
range(0, len(real), bs))

    @torch.no_grad()
    def _stats(self, batch_iter):
        s, ss, n = torch.zeros(2048, dtype=torch.float64),
torch.zeros(2048, 2048, dtype=torch.float64), 0
        for x in batch_iter:
            f = self.inception((x.clamp(0, 1).to(device) *
255).byte()).double().cpu()
            s += f.sum(0); ss += f.t() @ f; n += len(f)
        mu = s / n
        return mu, (ss - n * torch.outer(mu, mu)) / (n - 1)

    def __call__(self, batch_iter):
        mu, sigma = self._stats(batch_iter)
        tr_covmean = torch.linalg.eigvals(sigma @
self.sigma_r).sqrt().real.sum()
        return float((mu - self.mu_r).square().sum() + sigma.trace() +
self.sigma_r.trace() - 2 * tr_covmean)

def sample_batches(sample_fn, n, bs=500):
    done = 0
    while done < n:
        b = min(bs, n - done)
        yield sample_fn(b)
        done += b

def timed(fn):
    sync(); t = time.time(); out = fn(); sync()
    return out, time.time() - t

t = time.time()
fid_scorer = FIDScorer(X_test)
print(f"Real Inception statistics ready ({len(X_test)} test images,
{time.time() - t:.1f}s)")
```

```
Real Inception statistics ready (10000 test images, 9.0s)
```

In [5]:
```
CFG = dict(
    z_dim=128, dim=128, bs=64, gen_iters=40000, n_critic=5,
    eval_every=5000, n_fid_curve=5000, n_fid=10000,
    wgan=dict(lr=5e-5, clip=0.01),                        # values from the
WGAN paper
    wgan_gp=dict(lr=1e-4, betas=(0.0, 0.9), lam=10.0),    # values from the
WGAN-GP paper
)
if QUICK:
    CFG.update(gen_iters=30, eval_every=15, n_fid_curve=128, n_fid=256)
results = {"config": CFG, "device": str(device), "gpu_name": f"{GPU_NAME}
(host {socket.gethostname()})"}
CFG
```

Out[5]:
```
{'z_dim': 128,
 'dim': 128,
 'bs': 64,
 'gen_iters': 40000,
 'n_critic': 5,
 'eval_every': 5000,
 'n_fid_curve': 5000,
 'n_fid': 10000,
 'wgan': {'lr': 5e-05, 'clip': 0.01},
 'wgan_gp': {'lr': 0.0001, 'betas': (0.0, 0.9), 'lam': 10.0}}
```

2. Architecture

We use the DCGAN-style networks from the WGAN-GP paper's CIFAR-10 experiments (DIM = 128).

- **Generator:** $z \in \mathbb{R}^{128} \sim \mathcal{N}(0, I)$ → linear → 4×4×512 → three transposed convs (4→8→16→32) with BatchNorm+ReLU → `tanh`.
- **Critic:** three stride-2 convs (32→16→8→4) with LeakyReLU(0.2) → linear → a single scalar. There is **no sigmoid**, because the critic estimates a Wasserstein distance, not a probability.
  - WGAN (clipping) critic: uses BatchNorm, as in the original paper.
  - WGAN-GP critic: **no BatchNorm**. The gradient penalty is applied to each sample separately, and batch normalisation would mix samples within a batch (the GP paper points this out).

In [6]:

```
class Generator(nn.Module):
    def __init__(self, z_dim=128, dim=128):
        super().__init__()
        self.dim = dim
        self.fc = nn.Sequential(nn.Linear(z_dim, 4 * 4 * 4 * dim),
nn.BatchNorm1d(4 * 4 * 4 * dim), nn.ReLU(True))
        self.net = nn.Sequential(
            nn.ConvTranspose2d(4 * dim, 2 * dim, 4, 2, 1), nn.BatchNorm2d(2
* dim), nn.ReLU(True),   # 8x8
            nn.ConvTranspose2d(2 * dim, dim, 4, 2, 1),
nn.BatchNorm2d(dim),     nn.ReLU(True),   # 16x16
            nn.ConvTranspose2d(dim, 3, 4, 2, 1), nn.Tanh())
# 32x32

    def forward(self, z):
        return self.net(self.fc(z).view(-1, 4 * self.dim, 4, 4))


class Critic(nn.Module):
    def __init__(self, dim=128, batchnorm=False):
        super().__init__()
        norm = (lambda c: nn.BatchNorm2d(c)) if batchnorm else (lambda c:
nn.Identity())
        self.net = nn.Sequential(
            nn.Conv2d(3, dim, 4, 2, 1), nn.LeakyReLU(0.2, True),
# 16x16
            nn.Conv2d(dim, 2 * dim, 4, 2, 1), norm(2 * dim),
nn.LeakyReLU(0.2, True),            # 8x8
            nn.Conv2d(2 * dim, 4 * dim, 4, 2, 1), norm(4 * dim),
nn.LeakyReLU(0.2, True),        # 4x4
            nn.Flatten(), nn.Linear(4 * 4 * 4 * dim, 1))

    def forward(self, x):
        return self.net(x).view(-1)

print("G params: %.2fM | D params: %.2fM" % (n_params(Generator()) / 1e6,
n_params(Critic()) / 1e6))
```

```
G params: 3.70M | D params: 2.64M
```

3 & 4. Training WGAN and WGAN-GP

Both models minimise the Kantorovich–Rubinstein dual estimate of the Wasserstein-1 distance:

$$\min_{G} \max_{\|D\|_L \le 1} \mathbb{E}_{x \sim p_r}[D(x)] - \mathbb{E}_{z}[D(G(z))]$$

The critic is updated $n_{critic} = 5$ times for each generator update. The two methods differ in how they enforce the 1-Lipschitz constraint:

	WGAN	**WGAN-GP**
Lipschitz constraint	clip every critic weight to [-0.01, 0.01]	penalty $\lambda\,\mathbb{E}_{\hat{x}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$, λ=10, with $\hat{x}$ on random interpolates between real and fake
Optimiser	RMSProp, lr 5e-5 (momentum-based optimisers are unstable with clipping)	Adam, lr 1e-4, β=(0, 0.9), linear lr decay
Extra	100 critic iterations for the first 25 generator steps and every 500th step (as in the paper)	–

Both run for the same number of generator iterations (40k) with batch size 64. Every 5k iterations we compute a quick FID on 5k samples to track progress. The final FID uses 10k samples, which is the same protocol as Part A.

In [7]:

```
def sample_real(bs):
    return X_train[torch.randint(0, len(X_train), (bs,), device=device)] *
2 - 1    # [0,1] -> [-1,1]

@torch.no_grad()
def generate(G, n):
    G.eval()
    x = (G(torch.randn(n, CFG["z_dim"], device=device)) + 1) / 2
# back to [0,1]
    G.train()
    return x

def gradient_penalty(D, real, fake):
    a = torch.rand(real.size(0), 1, 1, 1, device=device)
    x_hat = (a * real + (1 - a) * fake).requires_grad_(True)
    grad = torch.autograd.grad(D(x_hat).sum(), x_hat, create_graph=True)[0]
    return ((grad.flatten(1).norm(2, dim=1) - 1) ** 2).mean()


def train_wgan(kind):
    assert kind in ("wgan", "wgan_gp")
    path = f"{CKPT}/{kind}.pt"
    hp = CFG[kind]
    G = Generator(CFG["z_dim"], CFG["dim"]).to(device)
    D = Critic(CFG["dim"], batchnorm=(kind == "wgan")).to(device)
    if os.path.exists(path):
        ck = torch.load(path, map_location=device)
        G.load_state_dict(ck["G"]); D.load_state_dict(ck["D"]); print(f"
{kind}: loaded checkpoint")
        return G, D, ck["hist"]

    if kind == "wgan":
        optG = torch.optim.RMSprop(G.parameters(), lr=hp["lr"])
        optD = torch.optim.RMSprop(D.parameters(), lr=hp["lr"])
        scheds = []
    else:
        optG = torch.optim.Adam(G.parameters(), lr=hp["lr"],
betas=hp["betas"])
        optD = torch.optim.Adam(D.parameters(), lr=hp["lr"],
betas=hp["betas"])
        decay = lambda it: 1.0 - it / CFG["gen_iters"]
        scheds = [torch.optim.lr_scheduler.LambdaLR(o, decay) for o in
(optG, optD)]

    hist = {"it": [], "w_dist": [], "d_loss": [], "g_loss": [], "gp": [],
"fid_it": [], "fid": [], "fid_time": []}
    fixed_z = torch.randn(64, CFG["z_dim"], device=device)
    bs, eval_time = CFG["bs"], 0.0
    sync(); t0 = time.time()
    for it in range(1, CFG["gen_iters"] + 1):
        # ---- critic updates
        n_crit = 100 if (kind == "wgan" and (it <= 25 or it % 500 == 0))
else CFG["n_critic"]
        for _ in range(n_crit):
            real = sample_real(bs)
```

```
            with torch.no_grad():
                fake = G(torch.randn(bs, CFG["z_dim"], device=device))
            d_real, d_fake = D(real).mean(), D(fake).mean()
            loss_d = d_fake - d_real
            if kind == "wgan_gp":
                gp = gradient_penalty(D, real, fake)
                loss_d = loss_d + hp["lam"] * gp
            optD.zero_grad(set_to_none=True)
            loss_d.backward()
            optD.step()
            if kind == "wgan":
                for p in D.parameters():
                    p.data.clamp_(-hp["clip"], hp["clip"])
        # ---- generator update
        loss_g = -D(G(torch.randn(bs, CFG["z_dim"], device=device))).mean()
        optG.zero_grad(set_to_none=True)
        loss_g.backward()
        optG.step()
        for s in scheds: s.step()

        if it % 100 == 0 or it % CFG["eval_every"] == 0 or it ==
CFG["gen_iters"]:
            hist["it"].append(it); hist["w_dist"].append((d_real -
d_fake).item())
            hist["d_loss"].append(loss_d.item());
hist["g_loss"].append(loss_g.item())
            hist["gp"].append(gp.item() if kind == "wgan_gp" else 0.0)
        if it % CFG["eval_every"] == 0 or it == CFG["gen_iters"]:
            te = time.time()
            fid = fid_scorer(sample_batches(lambda b: generate(G, b),
CFG["n_fid_curve"]))
            eval_time += time.time() - te
            train_t = time.time() - t0 - eval_time
            hist["fid_it"].append(it); hist["fid"].append(fid);
hist["fid_time"].append(train_t)
            print(f"[{kind}] it {it:6d} | W-dist est. {hist['w_dist']
[-1]:7.3f} | G loss {loss_g.item():8.3f} "
                  f"| FID({CFG['n_fid_curve']}) {fid:7.2f} | train time
{train_t / 60:6.1f} min")
            if it % (CFG["eval_every"] * 2) == 0 or it == CFG["gen_iters"]:
                G.eval()
                with torch.no_grad():
                    show_grid((G(fixed_z) + 1) / 2, nrow=16, title=f"{kind}
@ {it} iterations (fixed z)", figsize=(14, 4))
                G.train()
    sync()
    hist["train_time_s"] = time.time() - t0 - eval_time
    torch.save({"G": G.state_dict(), "D": D.state_dict(), "hist": hist},
path)
    return G, D, hist
```

3. WGAN (weight clipping)

In [8]:
```
seed_everything(SEED)
G_wgan, D_wgan, hist_wgan = train_wgan("wgan")
```

```
[wgan] it   5000 | W-dist est.   1.539 | G loss    0.763 | FID(5000)  249.16
| train time    0.9 min
[wgan] it  10000 | W-dist est.   1.537 | G loss    0.762 | FID(5000)  213.95
| train time    1.7 min
```

wgan @ 10000 iterations (fixed z)



```
[wgan] it  15000 | W-dist est.   1.532 | G loss    0.759 | FID(5000)  187.74
| train time    2.5 min
[wgan] it  20000 | W-dist est.   1.182 | G loss    0.581 | FID(5000)  168.47
| train time    3.3 min
```

wgan @ 20000 iterations (fixed z)



```
[wgan] it  25000 | W-dist est.   1.040 | G loss    0.461 | FID(5000)  139.65
| train time    4.1 min
[wgan] it  30000 | W-dist est.   1.042 | G loss    0.457 | FID(5000)  123.32
| train time    4.9 min
```

wgan @ 30000 iterations (fixed z)



```
[wgan] it  35000 | W-dist est.   0.827 | G loss    0.335 | FID(5000)  102.99
| train time    5.7 min
[wgan] it  40000 | W-dist est.   0.678 | G loss    0.293 | FID(5000)   88.12
| train time    6.5 min
```

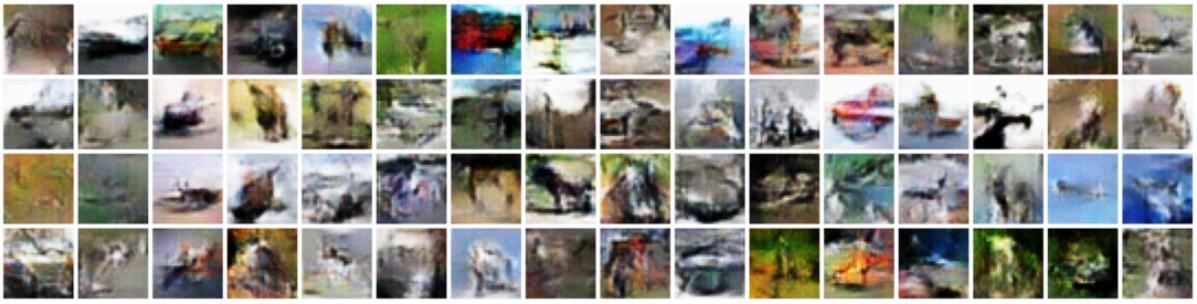


4. WGAN-GP (gradient penalty)

In [9]:
```
seed_everything(SEED)
G_gp, D_gp, hist_gp = train_wgan("wgan_gp")
```

```
[wgan_gp] it   5000 | W-dist est.   3.850 | G loss    3.094 | FID(5000)
131.41 | train time    1.3 min
[wgan_gp] it  10000 | W-dist est.   2.849 | G loss    3.493 | FID(5000)
93.15 | train time    2.7 min
```

wgan_gp @ 10000 iterations (fixed z)



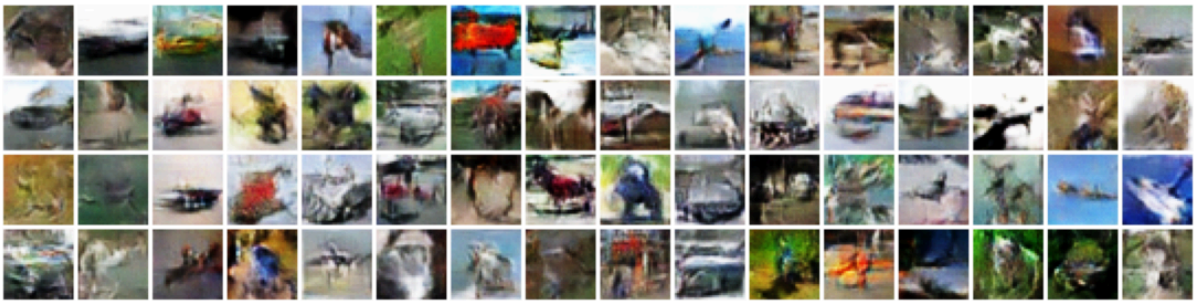
```
[wgan_gp] it  15000 | W-dist est.   3.091 | G loss    0.801 | FID(5000)
77.26 | train time    4.0 min
[wgan_gp] it  20000 | W-dist est.   2.239 | G loss    1.214 | FID(5000)
69.31 | train time    5.4 min
```

wgan_gp @ 20000 iterations (fixed z)



```
[wgan_gp] it  25000 | W-dist est.   2.932 | G loss    0.821 | FID(5000)
65.09 | train time    6.7 min
[wgan_gp] it  30000 | W-dist est.   2.601 | G loss    0.251 | FID(5000)
63.96 | train time    8.1 min
```

wgan_gp @ 30000 iterations (fixed z)



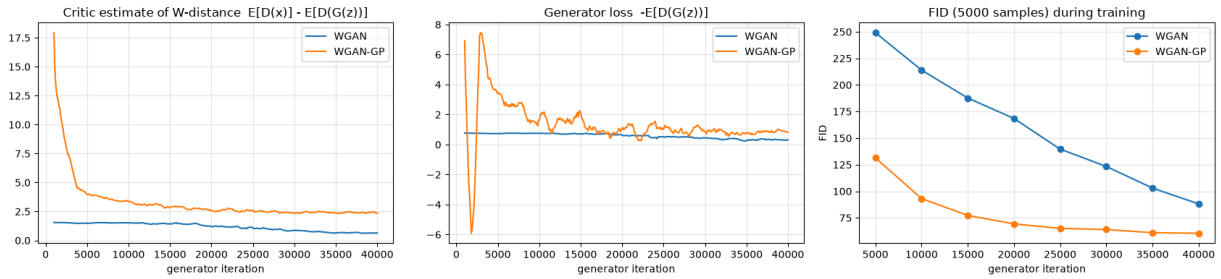
```
[wgan_gp] it  35000 | W-dist est.   2.210 | G loss    0.954 | FID(5000)
61.05 | train time    9.4 min
[wgan_gp] it  40000 | W-dist est.   2.316 | G loss    0.888 | FID(5000)
60.47 | train time   10.8 min
```



5. Training curves

In [10]:
```
def smooth(y, k=10):
    y = np.asarray(y, dtype=float)
    return np.convolve(y, np.ones(k) / k, mode="valid") if len(y) > k else
y

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
for h, name in [(hist_wgan, "WGAN"), (hist_gp, "WGAN-GP")]:
    w = smooth(h["w_dist"])
    ax[0].plot(h["it"][len(h["it"]) - len(w):], w, label=name)
    g = smooth(h["g_loss"])
    ax[1].plot(h["it"][len(h["it"]) - len(g):], g, label=name)
    ax[2].plot(h["fid_it"], h["fid"], "o-", label=name)
ax[0].set(title="Critic estimate of W-distance  E[D(x)] - E[D(G(z))]",
xlabel="generator iteration")
ax[1].set(title="Generator loss  -E[D(G(z))]", xlabel="generator
iteration")
ax[2].set(title=f"FID ({CFG['n_fid_curve']} samples) during training",
xlabel="generator iteration", ylabel="FID")
for a in ax: a.grid(alpha=0.3); a.legend()
plt.tight_layout(); plt.savefig(f"{FIG}/wgan_training_curves.png",
dpi=150); plt.show()
```



The critic's W-distance estimate is a useful training signal for both models. It falls steadily as the samples improve, which is one of the main selling points of WGANs compared with the standard GAN loss. Its scale differs a lot between the two models. With clipping the critic is restricted to a tiny weight box, so it can only estimate the distance up to an unknown (small) constant. WGAN-GP's estimate is on a more natural scale.

6. Random samples and final FID (10k samples vs. CIFAR-10 test set)

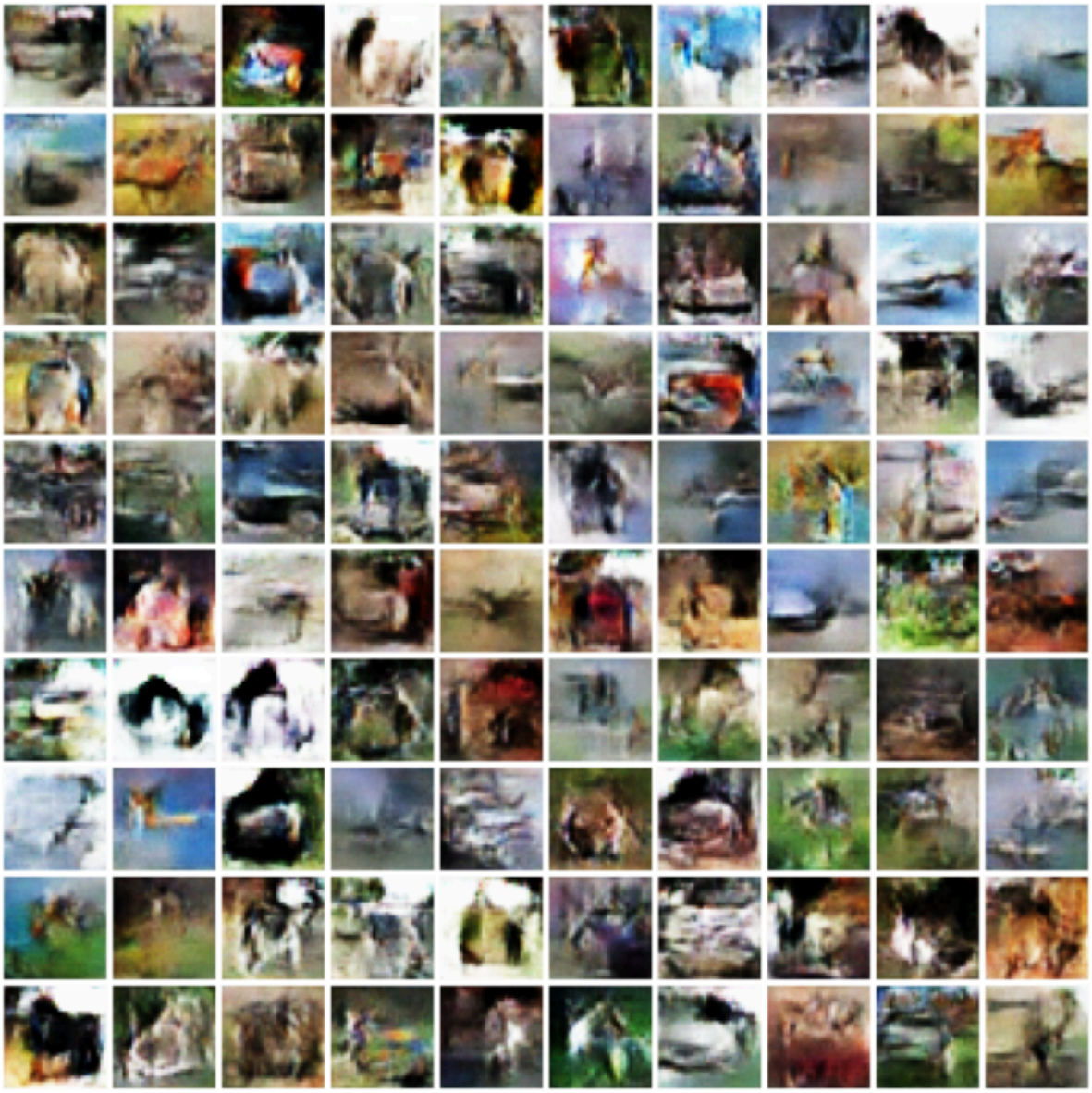
In [11]:

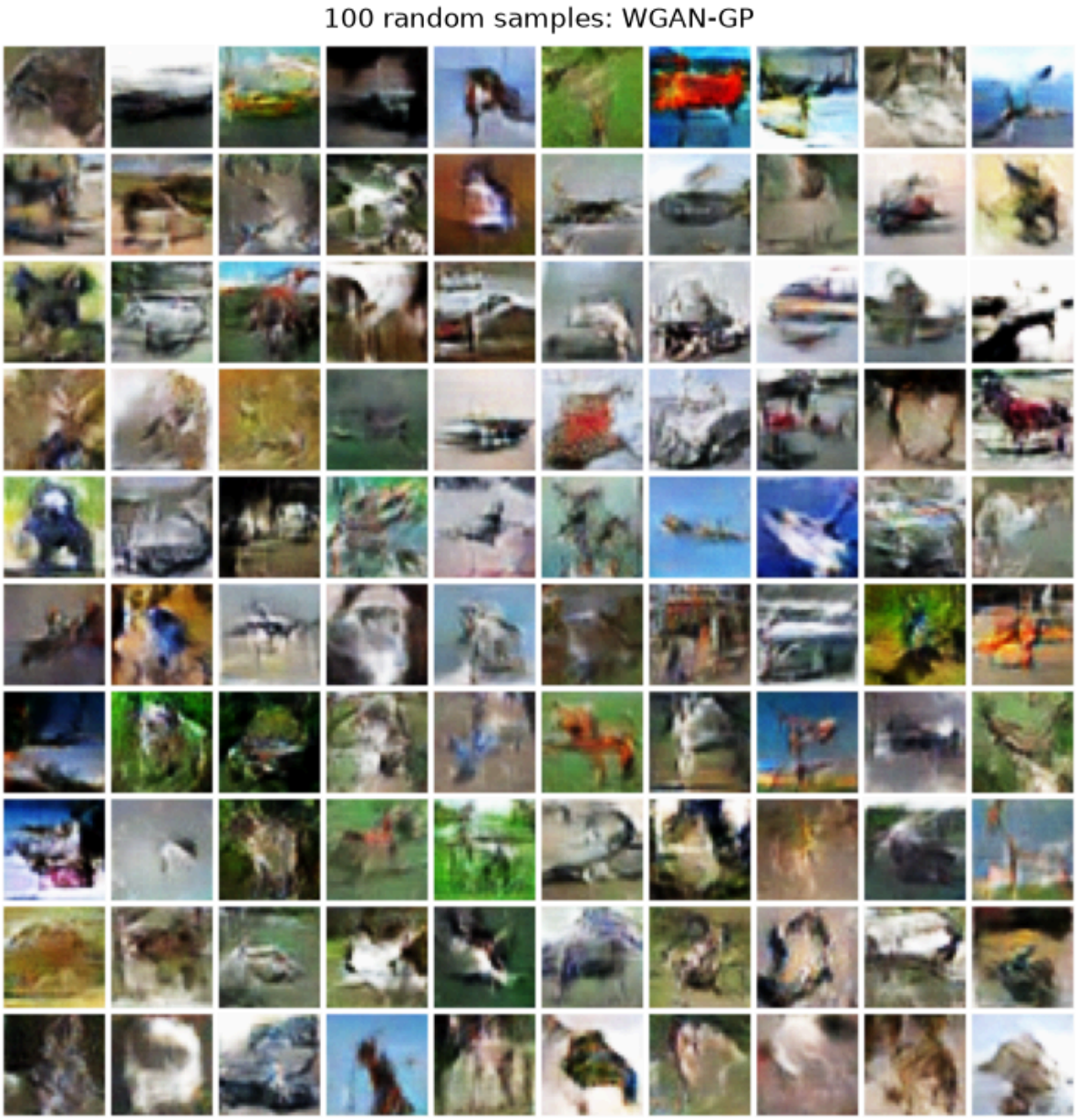
```
for kind, G in [("wgan", G_wgan), ("wgan_gp", G_gp)]:
    h = hist_wgan if kind == "wgan" else hist_gp
    seed_everything(SEED)
    fid = fid_scorer(sample_batches(lambda b: generate(G, b),
CFG["n_fid"]))
    _, t_sample = timed(lambda: [generate(G, 500) for _ in
range(max(CFG["n_fid"] // 500, 1))])
    D = D_wgan if kind == "wgan" else D_gp
    results[kind] = dict(fid=fid, fid_curve_it=h["fid_it"],
fid_curve=h["fid"],
                         best_curve_fid=min(h["fid"]),
train_time_s=h["train_time_s"],
                         sample_time_10k_s=t_sample * 10000 /
max(CFG["n_fid"], 500),
                         params_G=n_params(G), params_D=n_params(D),
gen_iters=CFG["gen_iters"])
    print(f"{kind:8s}: FID = {fid:.2f} | training time {h['train_time_s'] /
60:.1f} min "
          f"| 10k samples in {results[kind]['sample_time_10k_s']:.2f}s")

seed_everything(SEED)
show_grid(generate(G_wgan, 100), nrow=10, path=f"
{FIG}/wgan_samples_100.png",
          title="100 random samples: WGAN (weight clipping)", figsize=(9,
9))
seed_everything(SEED)
show_grid(generate(G_gp, 100), nrow=10, path=f"
{FIG}/wgangp_samples_100.png",
          title="100 random samples: WGAN-GP", figsize=(9, 9))
json.dump(results, open(f"{RES}/partB_results.json", "w"), indent=2)
```

```
wgan    : FID = 85.52 | training time 6.5 min | 10k samples in 0.04s
wgan_gp : FID = 57.86 | training time 10.8 min | 10k samples in 0.04s
```

100 random samples: WGAN (weight clipping)





Latent-space interpolation (WGAN-GP)

As a sanity check that the generator has learned a smooth mapping rather than memorising images, we decode linear interpolations between random latent vectors.

In [12]:

```
seed_everything(1)
z0, z1 = torch.randn(8, CFG["z_dim"], device=device), torch.randn(8,
CFG["z_dim"], device=device)
alphas = torch.linspace(0, 1, 10, device=device)
zs = torch.cat([(1 - a) * z0 + a * z1 for a in alphas]).view(10, 8,
-1).transpose(0, 1).reshape(-1, CFG["z_dim"])
G_gp.eval()
with torch.no_grad():
    show_grid((G_gp(zs) + 1) / 2, nrow=10, path=f"
{FIG}/wgangp_interpolation.png",
              title="WGAN-GP: linear interpolation in z (each row)",
figsize=(9, 7.5))
```



In [13]:
```
df = pd.DataFrame({
    "WGAN (clip)": {"FID ↓ (10k)": results["wgan"]["fid"], "Best FID during
training (5k)": results["wgan"]["best_curve_fid"],
                    "Train time (min)": results["wgan"]["train_time_s"] /
60, "Sampling 10k imgs (s)": results["wgan"]["sample_time_10k_s"],
                    "G params (M)": results["wgan"]["params_G"] / 1e6},
    "WGAN-GP": {"FID ↓ (10k)": results["wgan_gp"]["fid"], "Best FID during
training (5k)": results["wgan_gp"]["best_curve_fid"],
                "Train time (min)": results["wgan_gp"]["train_time_s"] /
60, "Sampling 10k imgs (s)": results["wgan_gp"]["sample_time_10k_s"],
                "G params (M)": results["wgan_gp"]["params_G"] / 1e6}}).T
print("WGAN-GP better than WGAN:", results["wgan_gp"]["fid"] <
results["wgan"]["fid"])
df.round(2)
```

```
WGAN-GP better than WGAN: True
```

Out[13]:

	FID ↓ (10k)	Best FID during training (5k)	Train time (min)	Sampling 10k imgs (s)	G params (M)
WGAN (clip)	85.52	88.12	6.53	0.04	3.7
WGAN-GP	57.86	60.47	10.78	0.04	3.7

Observations (Part B)

- **WGAN-GP clearly beats weight-clipped WGAN** on FID, and its FID curve falls much faster. This matches the WGAN-GP paper. Weight clipping restricts the critic to a very simple set of functions (most weights end up stuck at ±c), so the gradients it gives the generator are poor and training is slow. The gradient penalty enforces the Lipschitz constraint softly, only where it matters (between the real and fake distributions), which leaves the critic expressive and lets us use Adam.
- **Samples:** the WGAN-GP samples have sharper edges, more varied colours, and recognisable object/background layouts (horses, cars, ships, birds against the sky). The WGAN samples are noisier and blotchier. Neither model collapsed to a few modes: the 100-sample grids show a wide variety of colours and scenes.
- **Cost:** the gradient penalty needs a second backward pass (a gradient of a gradient), so each critic step of WGAN-GP is slower than with clipping. Sampling costs the same for both, a single generator forward pass, which takes well under a second for 10k images.